Java Persistence API
Object Relational Mapping

Databases 2

Application data integration

- Most web and mobile applications store and retrieve data from a **DBMS**
- Often the database already exists, independently of the application
- Converting data between a relational database and object-oriented code is unnecessarily complex
- Developers spend a lot of **repetitive code** converting rows/columns ↔ objects

E.g., in JDBC → opening connections, creating statements, processing result sets

Object Model vs. Relational Model

- Bridging the gap between objects and relational data is called **Object-Relational Mapping (ORM)**
- ORM maps concepts from one model to the other
- The main challenge is **impedance mismatch**: some concepts in one model have no direct equivalent in the other
- A **mediator** is needed to **automatically transform** data between objects and tables

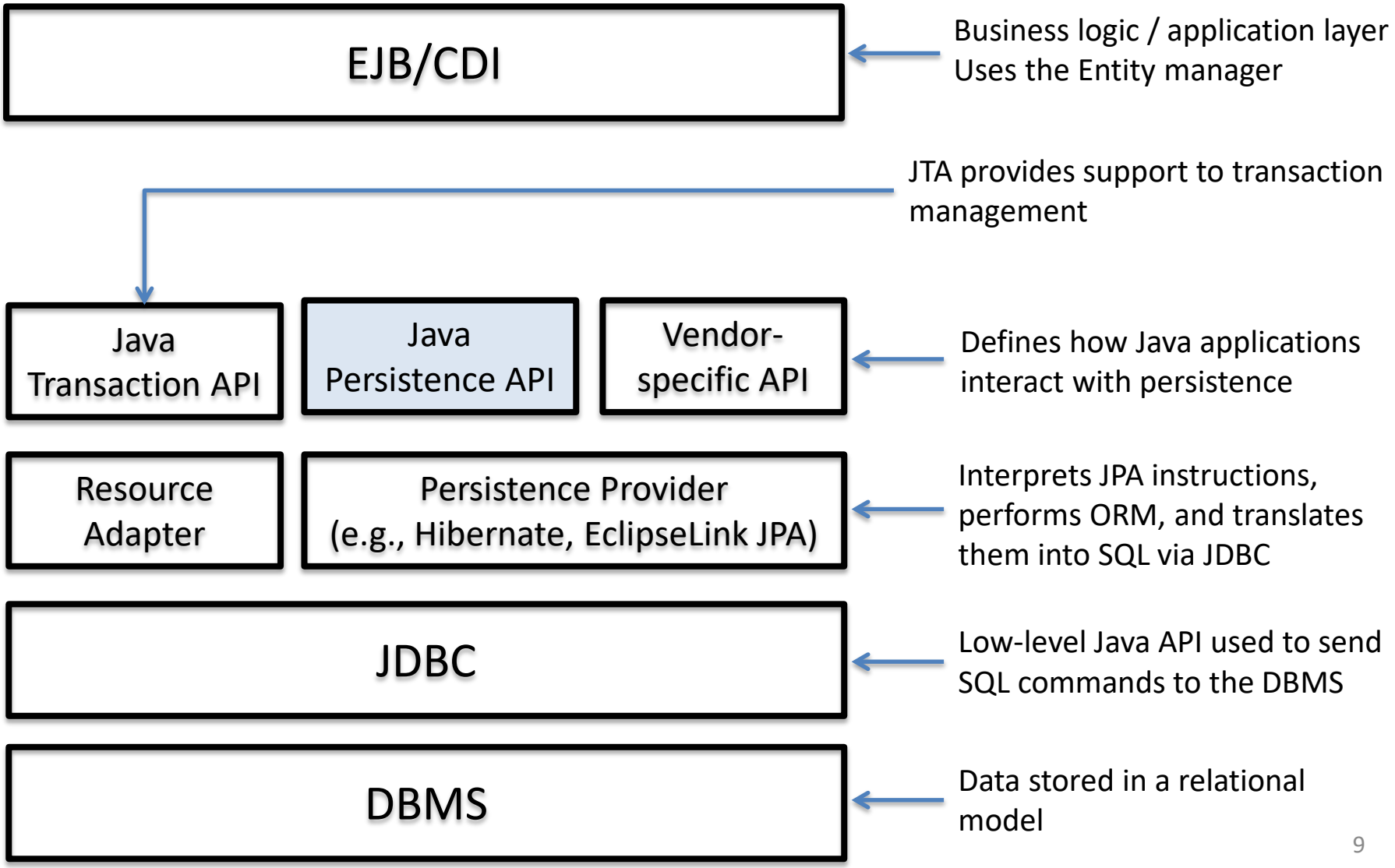
Differences between OM and RM

Object Oriented Model (Java)	Relational Model
Objects, classes	Tables, rows
Attributes, properties	Columns
Identity (physical memory address)	Primary key
Reference to other entity	Foreign key
Inheritance/Polymorphism	Not supported
Methods	Stored procedures, triggers
Code is portable	Not necessarily portable (depending on the vendor)

Java Persistence API

- **Java Persistence API (JPA)** bridges the gap between object-oriented domain models and relational database systems
  - it provides a POJO (Plain Old Java Object) persistence model for object-relational mapping
- Builds on earlier approaches such as
  - JDO: Java data Objects (JDO 2.0: JSR 243)
  - JDBC: Java database connectivity (JDBC 3.0 API)
- Standardized under **JSR 317**
- Can be used inside **Java EE / EJB containers** or in **plain Java SE** applications
- Works with or without an application server

JPA Architecture



JPA in a nutshell

- Java Persistence API main features:
  - **POJO Persistence:** any existing non-`final` object with a default constructor can be persisted
  - **Non-intrusiveness:** the persistence API exists as a separate layer from the persistent objects, i.e., the mapped objects are not aware of the persistence layer
  - **Object queries:** a powerful query framework (JPQL) allows querying objects and their relationships rather than tables and foreign keys

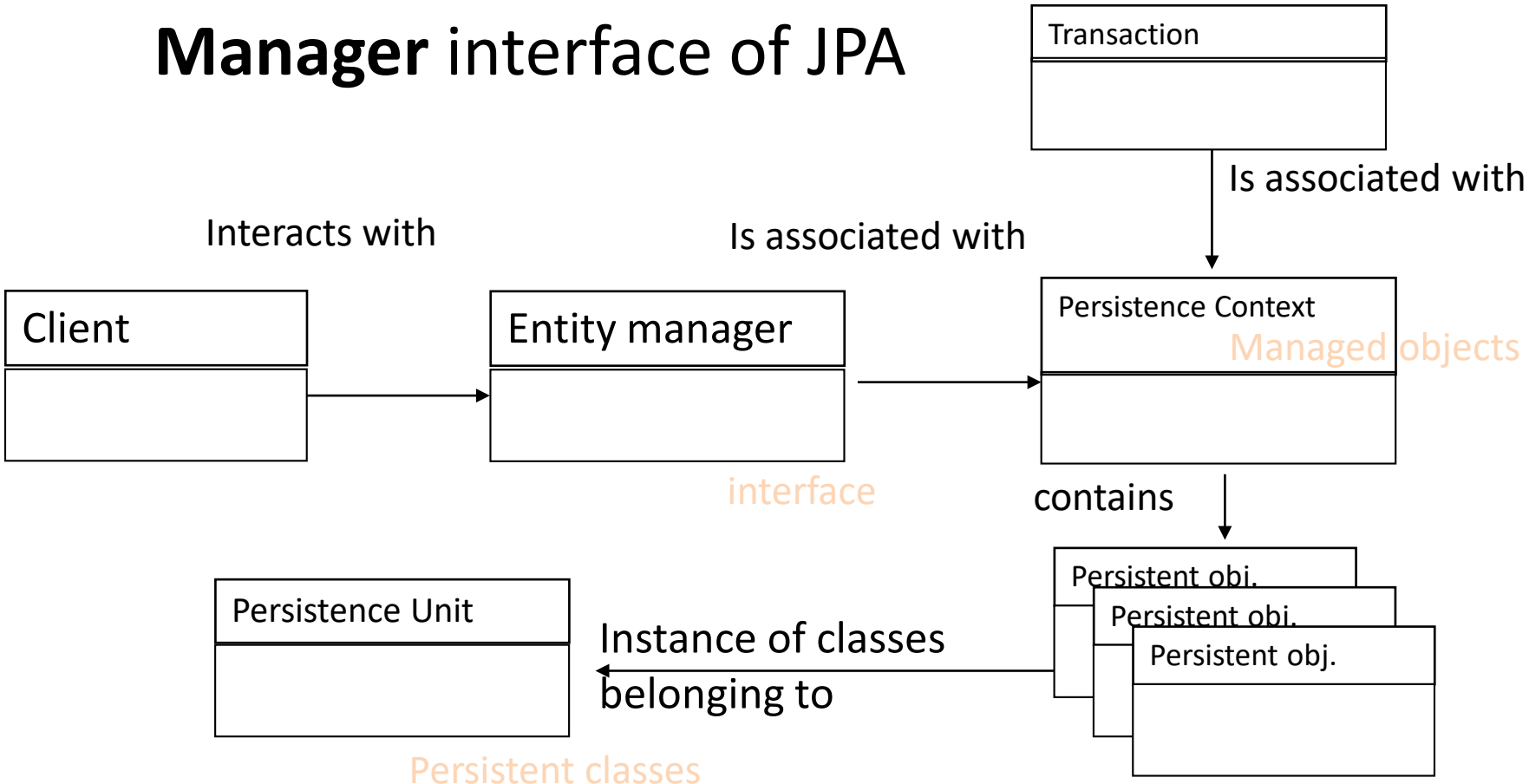
Advantage of JPA: database independence → code becomes more portable

JPA main concepts

- Entity: a class (JavaBean) representing a collection of *persistent objects* mapped onto a relational table
- Persistence Unit: the set of all classes that are persistently mapped to one database (analogous to a db schema)
- Persistence Context: the set of all *managed* objects of the entities defined in the persistence unit (analogous to the db instance in use)
- Managed entity: an entity part of a persistence context for which the changes of the state are tracked
- **Entity manager**: the interface for interacting with a Persistence Context
- Client: a component that can interact with a Persistence Context, indirectly through an Entity Manager (e.g., an EJB component)

How to work with entities

- Entities are accessed through the **Entity Manager** interface of JPA



Entity Manager interface

- The Entity Manager exposes all the operations needed to synchronize the managed entities in the persistence context to the database

Method signature	Description	
`public void persist(Object entity);`	Persists an entity instance in the database	Java object → database tuple
`public <T> T find(Class<T> entityClass, Object primaryKey);`	Finds an entity instance by its primary key	DB tuple → Java object
`public void remove(Object entity);`	Removes an entity instance from the database	Delete from DB – object still exists in memory
`public void refresh(Object entity);`	Reloads the entity instance from the database	Database → Overwrites Java object
`public void flush();`	Writes the state of entities to the database immediately	Synchronize managed state → Database

Some examples

See slides 05-3-3JPA-EM-compact.pdf

```
Employee employee = entityManager.find(Employee.class, 1);
        //retrieve employee with id=1 – this instance is now managed
employee.setName("Updated Name");   //changes are made to the entity
entityManager.flush(); // Synchronizes changes immediately
…

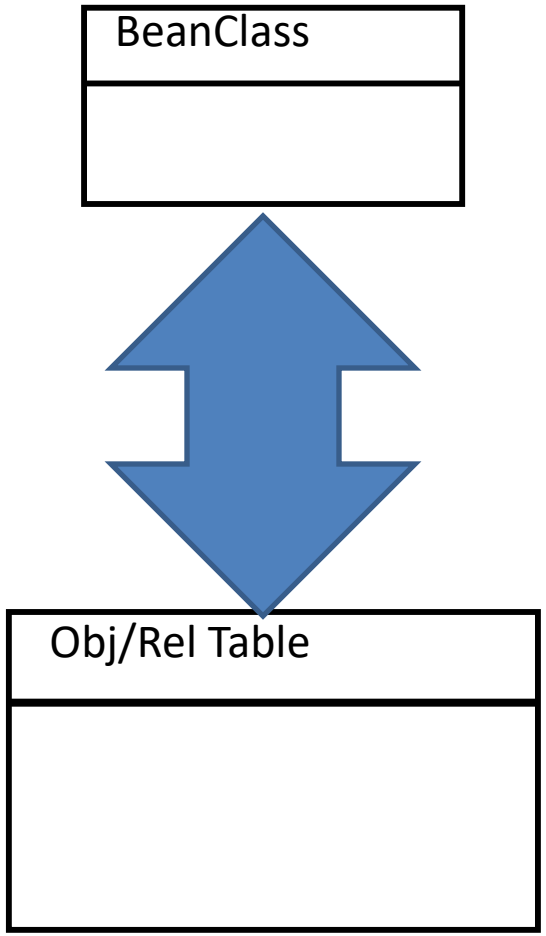
entityManager.remove(employee); //removed, no longer managed

Employee newEmployee = new Employee();
newEmployee.setId(2);
newEmployee.setName(“Marco Bianchi");
entityManager.persist(newEmployee);  //the new employee is now managed
```

Entity

- A Java Bean (POJO Plain Old Java Object) that gets associated to a tuple in a database
  - The class maps to the table schema
  - The objects map to the tuples
- The persistent counterpart of an entity has a life longer than that of the application
- The entity class must be associated with the database table it represents (mapping)
- An entity **can** enter in a managed state, where all the modifications to the object's state are tracked and automatically synchronized to the database

Entity Properties



- Identification (primary key)
- Nesting
- Relationship
- Referential integrity (foreign key)
- Inheritance

Entity (example)

= this class should be mapped
to a relational database table

Employee.java

```
@Entity
public class Employee {
    @Id private int id;
    private String name;
    private long salary;
    public Employee() {}
    public Employee(int id) { this.id = id; }
    public int getId() { return id; }
    public void setId(int id) { this.id = id; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public long getSalary() { return salary; }
    public void setSalary (long salary) { this.salary = salary; }
}
```

Java annotations are used to qualify the class as an entity

The entity instance is just a POJO

Entity constraints

- Entities must respect the following requirements:
  - The entity class must have a `public` or `protected` no-arg constructor
  - The entity class must not be `final`
  - No method or persistent instance variables of the entity class may be final
  - If an entity instance is to be passed by value as a detached object, the `Serializable` interface must be implemented (Serialization converts an object into a byte stream)

Entity Identification

- In database, objects and tuples have an identity (primary key)
- → an entity assumes the identity of the persistent data it is associated to
- Simple Primary key = persistent field of the bean used to represent its identity
- Composite Primary key = set of persistent fields used to represent its identity
- Remark: with respect to the POJOs, the persistent identity is a new concept. **POJOS do not have a durable identity**

Entity identification syntax

```
@Entity
public class Mission
   implements Serializable {

   @Id     = this is the PK
   private int id;
   private String city;
..
```

- `@Id` tags a field as the simple primary key
- Composite primary keys are denoted using the `@EmbeddedId` and `@IdClass` annotations

Entity: Identifier generation

- Sometimes, applications do not want to explicitly manage uniqueness of data values
  - The persistence provider can automatically generate an identifier for every entity instance of a given type
- This persistence provider's feature is called **identifier generation** and is specified by the `@GeneratedValue` annotation

Identifier generation options

- Applications can choose one of four different ID generation strategy

Id generation strategy	Description
`AUTO`	The persistence provider generates identifiers by using whatever strategy it wants
`TABLE`	Identifiers are generated according to a generator table
`SEQUENCE`	If the underlying DB supports sequences, the provider will use this feature for generating IDs
`IDENTITY`	If the underlying DB supports primary key identity columns, the provider will use this feature for generating IDs (e.g., AUTO-INCREMENT)

Identifier generation annotation

```
@Entity
public class Mission implements Serializable {

@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private int id;

private String city;
```

Generated by the DB when the tuple is created

- Whenever possible let the database generate the unique Ids

Attribute specifications

- Attributes can be qualified with properties that direct the mapping between POJOs and relational tables
- Mapping base and special types
  - Large objects
  - Enumerated types
    - As Java enumerations
    - As strings
  - Temporal types (DATE, TIME, TIMESTAMP)
    - Java temporal types
    - JDBC temporal types
- Specifying fetch policy
  - LAZY
  - EAGER (default)

```
@Entity
public class Mission implements
    Serializable {

    @Id
    @GeneratedValue(strategy =
    GenerationType.IDENTITY)
    private Integer id;

    @Temporal(TemporalType.DATE)
    private Date date;

    @Enumerated(EnumType.STRING)
    private MissionStatus status;

    @Basic(fetch=FetchType.LAZY)
    @Lob
    private byte[] photo;
}
```

- Specifying LAZY policy makes the attribute value remain empty when the object is retrieved, until the attributed is accessed explicitly (→ use for LOBs only)

Object to table mapping

- By default entities are mapped to tables with the same name and their fields to columns with the same names
- Defaults can be overridden
- Some annotations (e.g., `@column`) may have attributes used for generating the database schema from the entity classes (e.g., `nullable, unique, length`)
- Entities may also have attributes that are not persisted (`@Transient`)

```
@Entity @Table(name="T_BOOKS")
public class Book {

 @Column(name="BOOK_TITLE",
   nullable=false)
  private String title;

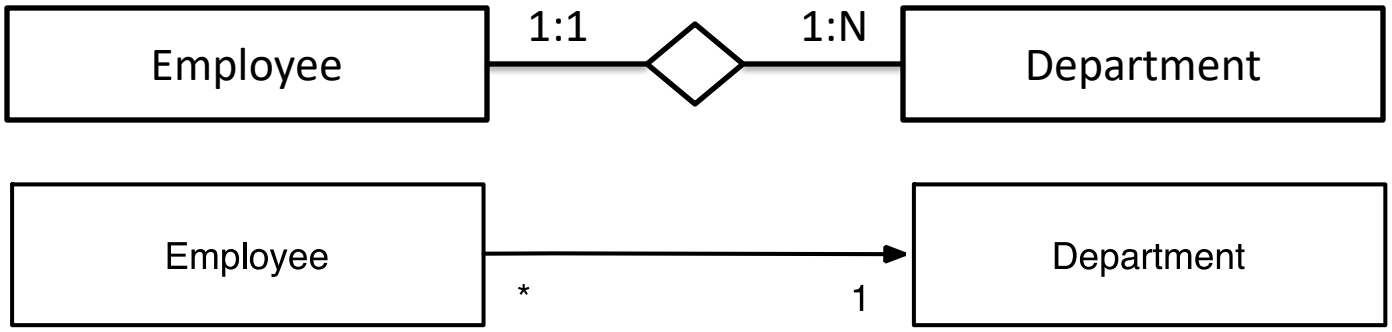
  @Enumerated(EnumType.STRING)
  private CoverType coverType;

  @Temporal(TemporalType.DATE)
  private Date publicationDate;

  @Transient
  private BigDecimal discount;
}
```

Entities and **relationships**

- If entities contained only simple persistent state, the issue of ORM would be a trivial one
- In fact, most entities need to be able to have **relationships** with other entities
- NOTE: there is an ambiguity in the meaning of "relationship" in the Entity-Relationship conceptual model and in the Object Model
- From now on we will use relationship in the sense of the Object Model



Relationship concepts

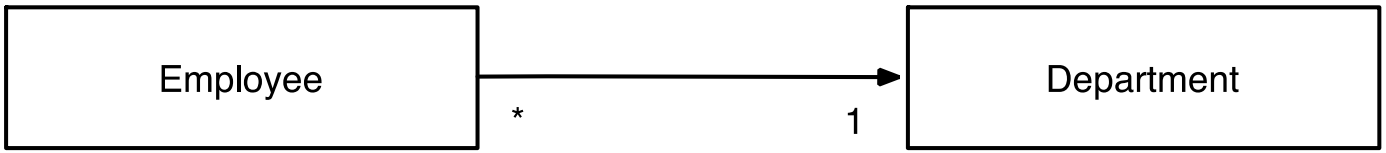
- Every (OM) relationship has four characteristics:
  - **Directionality**: each of the two entities may have an attribute that enables access to the other one
  - **Role**: each entity in the relationship is said to play a role with respect to one direction of access
  - **Cardinality**: the number of entity instances that exist on each side of the relationship
  - **Ownership**: one of the two entity in the relationship is said to own the relationship
    - Needed for code generation

Directionality

- Each entity in the relationship may have a **reference** to the other entity:
  - When each entity refers to the other, the relationship is **bidirectional**
  - If only one entity has a reference to the other, the relationship is said to be **unidirectional**
  - The reference can be single-valued or set-valued (collection of references)
- All relationships in JPA are unidirectional
  - A **bidirectional** relationship is intended as a "matched" **pair of unidirectional mappings**
  - "Matching" must be declared explicitly (via the dedicated attribute **`mappedBy`**)

Roles

- Based on directionality, one entity plays the role of **source** and one the role of **target**
- In the relationship from `Employee` to `Department`
  - `Employee` is the source entity
  - `Department` is the target entity

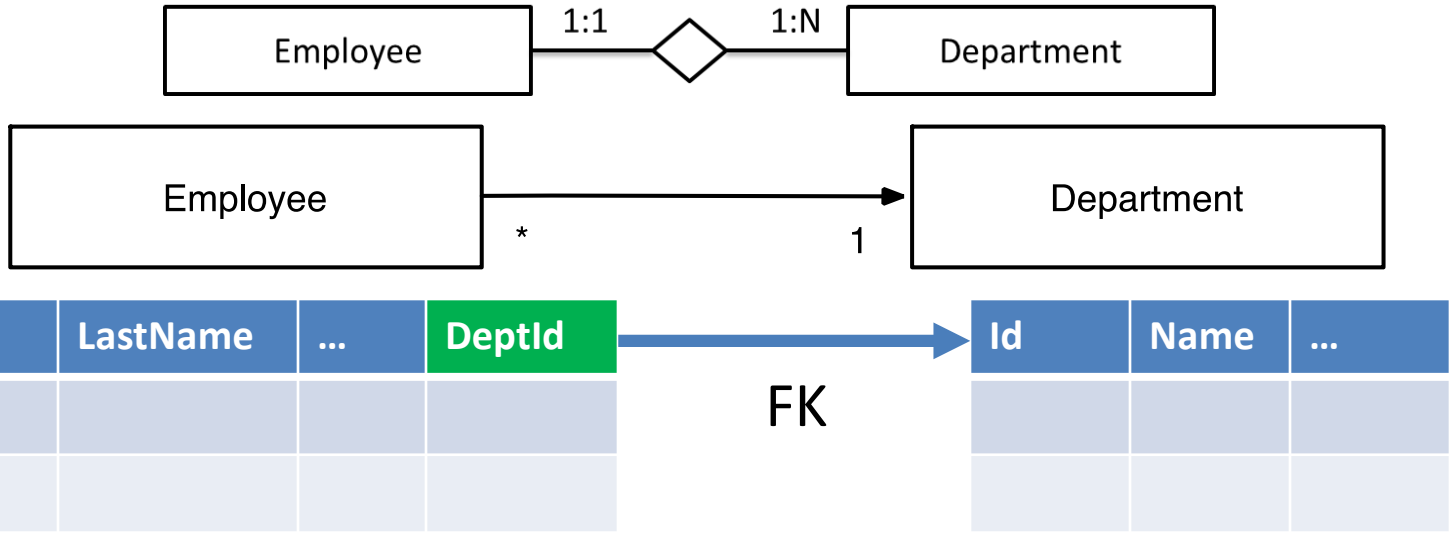


Cardinality

- Each role in the relationship has its own cardinality. This leads to four possible combinations:
  - **Many-to-one:** many source entities, one target entity
  - **One-to-many:** one source entity, many target entities
  - **One-to-one:** one source entity, one target entity
  - **Many-to-many:** many source entities, many target entities
- Remember: bidirectional relationships (the E-R way) are just pairs of matched unidirectional relationships with swapped source and target entities
- A to-one relationship implies that the source entity has **one reference** to the target entity,
- A to-many relationship implies that the source entity has a **multiple references** (*collection*) to the target entity

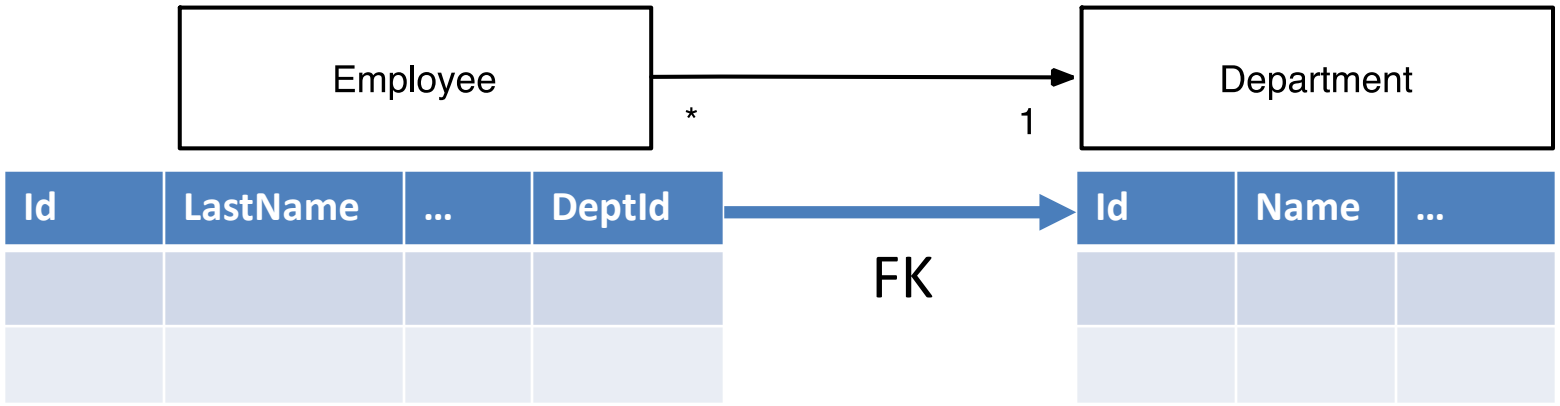
Ownership (1/2)

- In the database, relationships are implemented by a foreign key column that refers to the key of the referenced table
  - In JPA, such a column is called **join column**
- Example:
  - Many-to-one unidirectional relationship between Employee and Department
  - The underlying Employee table has a FK column containing the Department primary key



Ownership (2/2)

- In 1:N and 1:1 relationships, one of the two entities will have the FK column in its table
  - This entity is called the **owner** of the relationship and its side is called the **owning side**
  - **Owner** = the entity that provides the physical representation of the relationship (the join column)
- Example: **Employee is the owner** of the relationship
- Ownership is important because the annotations that define the physical mapping of the relationship (FK column) are specified in the owner side of the relationship



Ownership (new)



OWNING SIDE
(has the FK → JOIN COLUMN)

INVERSE SIDE
(can be used for navigation)

The relation can be updated only on this side:

```
Employee emp = em.find(Employee.class, 10);
Department newDept = em.find(Department.class, 3);
```

```
emp.setDepartment(newDept);    // owner side update
em.persist(emp);
```

JPA →
UPDATE EMPLOYEE
SET department_id = 3
WHERE id = 10;

```
dept.getEmployees().add(emp);   // inverse side update
em.persist(dept);
```

JPA →
No updates

Possible relationship mappings

- 1:N
  - Bidirectional
  - Unidirectional one-to-many & many-to-one
- 1:1
  - Bidirectional
  - Unidirectional
- N:M
  - Bidirectional
  - Unidirectional
- Whether to go for uni- or bi-directionality depends on the application
- If the relationship is accessed from one entity to the other and vice versa, use bi-directionality otherwise use uni-directionality
- NOTEs:
  - Bi-directionality can be also implemented with unidirectional relationship mapping + set-valued query
    - E.g., Employee knows the department
    - "SELECT e FROM Employee e WHERE e.department = :dept"
  - Performance considerations may prompt for the definition of bidirectional mappings even if only one direction is used

1:N bidirectional

- Expressed with (`@ManyToOne` + `@OneToMany` + `mappedBy`)
- The many-to-one mapping direction is defined by annotating the entity that participates with multiple instances with the `@ManyToOne` annotation
- The entity that contains the **`@ManyToOne`** annotation is the **owner** of the relationship because its underlying table stores the FK to the referenced table
- The **`@JoinColumn`** annotation can be used to specify the FK column of underlying table (defaults to the data member name)

`@ManyToOne` annotation in `Employee.java`

```
@Entity
public class Employee {
    @Id private int id;
    @ManyToOne                  //many employees belong to a dept
    @JoinColumn(name="dept_fk")   //column name in the db
    private Department dept;      //Java name of the field
    ...
}
```

1:N bidirectional, continued

- To achieve bi-directionality, the one-to-many mapping direction must be specified too
- This is done by including a `@OneToMany` annotation in the entity that participates with one instance
- The `@OneToMany` annotation is placed on a **collection** data member and comprises a **`mappedBy`** element to indicate the property that implements the inverse of the relationship
- NOTE: In a one-to-many mapping the owner of the relationship is still the entity that participates with multiple instances

`@OneToMany` annotation in `Department.java`

```
@Entity
public class Department {
    @Id private int id;
    @OneToMany(mappedBy="dept")  //if bidirectionoal
    private Collection<Employee> employees;
    ...
}
```

The attribute on the target entity that implements the inverse of the relationship

Uni-directional 1:N: many-to-one

- Sometimes applications require the access to relationships only along one direction
- In this case the bi-directional mapping is not necessary
- The many-to-one direction case is trivial
- Use just the same annotations as in the bidirectional case and skip the inverse mapping

```
@Entity
public class Employee {
    @Id private int id;

    @ManyToOne
    @JoinColumn(name="dept_fk")
    private Department dept;
    ...
}
```

Uni-directional 1:N: one-to-many

- The one-to-many direction is less trivial
- Two alternatives are viable performance-wise
  - Map the relationship as in the **bi-directional** case and use only the one-to-many direction
  - Do not map the collection attribute in the entity that participates with one instance and **use a query** instead to retrieve the correlated instances, relying on the inverse (many-to-one) relationship direction mapping

```
List<Employee> emps =
entityManager.createQuery(
  "SELECT e " +
  "FROM Employee e " +
  "WHERE e.dept.id = :deptId",
  Employee.class)
.setParameter("deptId", "DEIB")
.getResultList();
```

- JPQL query language

Normally, map both directions.
If you decide to not map the collection side, when you need it, you specify the query

"I own the join column"

"if you want this relationship go on the other side and look in this attribute"

`@joincolumn` **VS** `mappedBy`

- The annotation `@JoinColumn` indicates the FK column that implements the relationship in the database; such annotation is normally inserted in the entity **owner** of the relationship (i.e., the one mapped to the table that actually contains the FK column)
  - Used to drive the generation of the SQL code to extract the correlated instances
- The `mappedBy` attribute indicates that "this side" is the inverse of the relationship, and the owner resides in the "other" related entity
  - Used to specify **bidirectional** relationships

"Don't create a bridge table"
(if the relationship is not N:M)

Why specifying `MappedBy`?

- In absence of the `mappedBy` parameter the default JPA mapping (created when the database is generated from the JPA entities) uses a bridge table (as for N:M relationships)
- The purpose of the `mappedBy` parameter is to instruct **JPA** NOT to create a bridge table as the relationship is already being **mapped by** a FK in the opposite entity of the relationship

One-to-one mapping (1/2)

- In a one-to-one mapping the owner can be either entity, depending on the database design
- A one-to-one mapping is defined by annotating the owner entity (the one with the FK column) with the `@OneToOne` annotation
  - Owner: Employee

@OneToOne annotation in Employee.java

```
@Entity
public class Employee {
    @Id private int id;
    @OneToOne
    private ParkingSpace parkingSpace; // use join column default name
    ...
}
```

Join column default name

- If no @JoinColumn annotation accompanies the to-one mapping, a default column name will be assumed.
- The name that is used as the default is formed from a combination of both the source and target entities.
  - It is the name of the relationship attribute in the source entity, which is Employee in our example, plus an underscore character (_), plus the name of the primary key column of the target entity.
  - For example, *parkingspace_id*
- If this is not actually the name of the column, the @JoinColumn annotation must be defined to override the default
  - *@OneToOne*
    @JoinColumn(name = "P_SPACE_FK")

One-to-one mapping (2/2)

- If the one-to-one mapping is **bidirectional**, the inverse side of the relationship needs to be specified too
- In the other non-owner entity, the `@OneToOne` annotations must come with the `mappedBy` element
- The presence of `mappedBy` tells JPA that the foreign key constraint is in the table mapping the OTHER entity

@OneToOne annotation (inverse side) in ParkingSpace.java

```
@Entity
public class ParkingSpace {
    @Id private int id;
    @OneToOne(mappedBy="parkingSpace")   //If bidirectional
    private Employee employee;
    ...
}
```

Many-to-many mappings (1/2)

- In a many-to-many mapping **there is no FK column**
- Such a mapping is implemented by means of a **join table** (aka **bridge table**)
- Therefore, we can arbitrarily specify as owner **either entity**
  - For example, Employee

@ManyToMany annotation in Employee.java

```
@Entity
public class Employee {
    @Id private int id;
    @ManyToMany
    private Collection<Project> projects; // use default join table
        ...
}
```

Tell the code generator where the join (bridge) table is

Many-to-many mappings (2/2)

- If the many-to-many mapping is bidirectional, the inverse side of the relationship needs to be specified too
- In the other entity, the `@ManyToMany` annotation must come with the `mappedBy` element

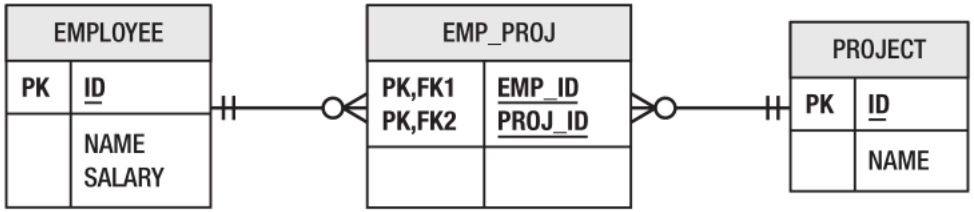
@ManyToMany annotation (inverse side) in Project.java

```
@Entity
public class Project {
    @Id private int id;
    @ManyToMany(mappedBy="projects")    // if bidirectional
    private Collection<Employee> employees;
    ...
}
```

Join Table

- The logical model of a N:M relationships requires a bridge table (JOIN table in the JPA terminology)
- The **non-default mapping** of the entity to the bridge table is specified via annotation

```
@Entity
public class Employee {
 @Id private long id;
 private String name;
 @ManyToMany
 @JoinTable(name="EMP_PROJ",
  joinColumns=@JoinColumn(name="EMP_ID"),
  inverseJoinColumns=@JoinColumn(name="PROJ_ID"))
 private Collection<Project> projects;
// ...
}
```



- When no @JoinTable is specified, then a **default name** `<Owner>_<Inverse>` is assumed, where `<Owner>` is the name of the owning entity, and `<Inverse>` is the name of the inverse or non-owning entity (`Employee_Project` in the example, with default columns Employee_id and Project_id)

Relationship fetch mode (1/3)

- **When loading** an entity, it is questionable if related entities are to be fetched & loaded too
  - Performance can be optimized by deferring data fetch until the time when they are needed
- This design pattern is called **lazy** (opposite= **eager**)
- Loading policy can be expressed specifying **fetch mode** for relationships (as seen for attributes of LOB type)
- At relationship level, lazy loading can greatly enhance performance because it can reduce the amount of SQL that is executed if correlated instances are seldom accessed by the application

Flexibility in balancing performance and data availability

Relationship fetch mode (2/3)

- When the **fetch mode** is not specified, by **default**:
  - A single-valued relationship is fetched **eagerly**
  - Collection-valued relationships are loaded **lazily**
- In case of bidirectional relationships, the fetch mode might be lazy on one side but eager on the other
  - Quite common situation, relationships are often accessed in different ways depending on the direction from which navigation occurs
- NOTE: Consider lazy loading as the most appropriate mode for all relationships, because if an entity as **many** single-valued relationships that are **not all used by applications**, the default eager mode may incur performance penalties

Lazy loading of the parkingSpace attribute

```
@Entity
public class Employee {
    @Id private int id;
    @OneToOne(fetch=FetchType.LAZY)
    private ParkingSpace parkingSpace;
    ...
}
```

Example

```
class Person {
   @OneToOne Address address;
   @ManyToOne Department department;
   @ManyToOne Company company;
   @OneToOne Profile profile;
}
```

Default = EAGER

Every time you load a **Person**,
JPA also loads: the address, the department, the company, the profile…
even if the application does not need them.

This results in:
larger queries,
unnecessary joins,
more data being transferred

Relationship fetch mode (3/3)

- The directive to lazily fetch an attribute is meant only to be a **hint** to the persistence provider
  - The provider is not required to respect the request because the behavior of the entity is not compromised if the provider decides to eagerly load
- The converse is **not true** because specifying that an attribute be eagerly fetched might be critical to access the entity once detached (i.e., no longer managed)

Cascading operations (1/4)

- By default, every Entity Manager operation applies **only to the entity supplied as an argument to the operation**
  - The operation **will not** cascade to other entities that have a relationship with the entity that is being operated on
- For some operations (e.g., `remove()`) this is usually the **desired behavior**

```
Employee emp = new Employee();
Department dept = new Department();
emp.setDepartment(dept);
em.persist(emp);
```

Only emp is persisted, not dept

```
em.remove(dept);
```

It does not remove the associated Employees. It deletes only the Department (and it fails if there are foreign keys and database constraints).

Cascading operations (2/4)

- Some other operations usually require cascading, such as `persist()`
  - If an entity has a relationship to another (dependent, aka "child") entity normally the child entity must be persisted together with the father entity
- Example: One-to-Many unidirectional mapping between `Employee` and `Address`

Manually cascading

```
Employee emp = new Employee();
Address addr = new Address();
emp.setAddress(addr);
em.persist(addr);
em.persist(emp);
```

We would like to avoid explicit persisting the `Address` entity instance

Cascading operations (3/4)

- The `cascade` attribute is used to define when operations should be automatically cascaded across relationships
  - As the entity manager adds the `Employee` instance to the persistence context, it navigates the address relationship looking for a new `Address` entity to manage as well
  - The `Address` instance must be set on the Employee instance before invoking `persist()` on the employee object
  - If an `Address` instance has been set on the `Employee` instance and not persisted explicitly or implicitly via cascading, an error occurs

Enabling cascade persist

```
@Entity
public class Employee {
    @OneToMany(cascade=CascadeType.PERSIST)
    Address address;
}
```

```
Employee emp = new Employee();
Address addr = new Address();
emp.setAddress(addr);
em.persist(emp);
```

Cascading operations (4/4)

- The `cascade` attribute accepts several possible values specified in the `CascadeType` enumeration:
  - `PERSIST`, `REFRESH`, `REMOVE`, `MERGE` and `DETACH`
  - **`ALL` is a shorthand for declaring that all five operations should be cascaded**
- As for relationships, cascade settings are **unidirectional**
  - they must be explicitly set on both sides of a relationship, if the default behavior has to be overridden
- NOTE: there is **no default cascade type in JPA**.
  By default no operations are cascaded
- Cascading remove operations may generate rather inefficient SQL code (related entities are deleted one by one with multiple statements) and should be avoided in most practical cases

Orphan removal

- JPA supports an additional remove cascading mode specified using `orphanRemoval` in `@OneToOne` and `@OneToMany`
  - appropriate for **privately owned** parent-child relationship, in which every child entity is associated only to one parent entity through just one relationship (weak entity in the E-R terminology)
  - causes the child entity to be removed when the parent-child relationship is broken either by removing the parent or by setting to null the attribute that holds the related entity, or in the one-to-many case, by removing the child entity from the parent's collection

```
@OneToMany(mappedBy="dept", orphanRemoval = true)
private List<Employee> employees;
```

`em.remove(dept);`	`dept.setEmployees(null);`	`dept.getEmployees().remove(emp);`
Delete all employees	Delete all employees	Delete emp

Orphan removal

- Difference between `CascadeType.REMOVE` and `orphanRemoval=true`
  - invoking `setEmployees(null)` on `Department` (set the collection to null – i.e., simply remove the reference)
  - For orphan removal: the related `Employee` entities are removed from the database automatically (if they are managed and no longer referenced)
  - For remove cascade: the related `Employee` entities are **NOT** removed from the database automatically (they are removed only when the parent is deleted)
  - If the **orphan** object is not **in the managed state**, removal does not occur

em.remove(dept);	dept.setEmployees(null);	dept.getEmployees().remove(emp);
cascaded	NO → not a remove of dept	NO → not a remove of dept

Several advantages

- Focus on higher-level application logic
  - Developers work with objects instead of SQL
- Declarative programming with annotations
  - Mapping and behaviors are expressed clearly and concisely
- Standardized API
  - Applications can switch between providers (Hibernate, EclipseLink, …) with minimal changes
- Built-in transaction management support (JTA)
- Improved productivity, maintainability, and portability
  - Cleaner code, reduced duplication, and better long-term adaptability.

References

- JPA specifications, http://www.jcp.org/en/jsr/detail?id=317
- Pro JPA 2 in Java EE8, *M. Keith, M. Schincariol, Apress Media LLC*
- *Chapter 4 and 10*

